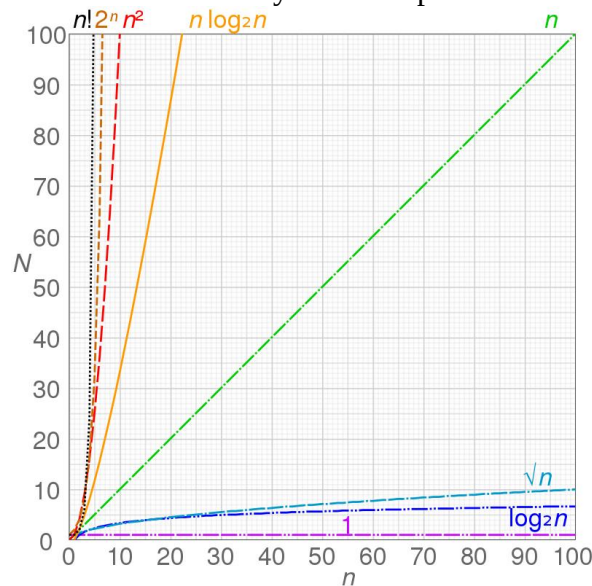


Chapter 8 NP Completeness [5]

In terms of computational complexity lets recall the function growth rate with respect to input and measure time taken to produce output. We can also recall easy and hard problems that is represented by graph.



Tractable and Intractable Problems, Complexity Classes

Generally, we think of problems that are solvable by polynomial-time algorithms as being **tractable**, or easy, and problems that require super polynomial time as being **intractable**, or **hard**.

The subject of this unit is an interesting class of problems, called the “**NP-complete**” problems, whose status is unknown.

No polynomial-time algorithm has yet been discovered for an NP-complete problem, nor has anyone yet been able to prove that no polynomial-time algorithm can exist for any one of them.

This so-called $P \neq NP$ question has been one of the deepest, most perplexing open research problems in theoretical computer science since it was first posed in 1971.

Several NP-complete problems are particularly tantalizing because they seem on the surface to be similar to problems that we know how to solve in polynomial time. In each of the following pairs of problems, **one is solvable in polynomial time** and the other is **NP-complete**, but the difference between problems appears to be slight:

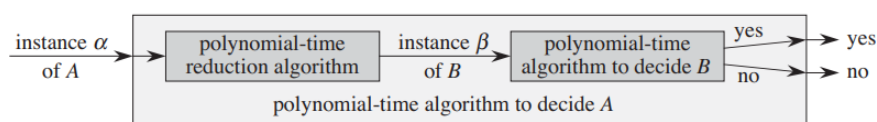
Shortest vs. longest simple paths

Euler tour vs. hamiltonian cycle (relate by tsp)

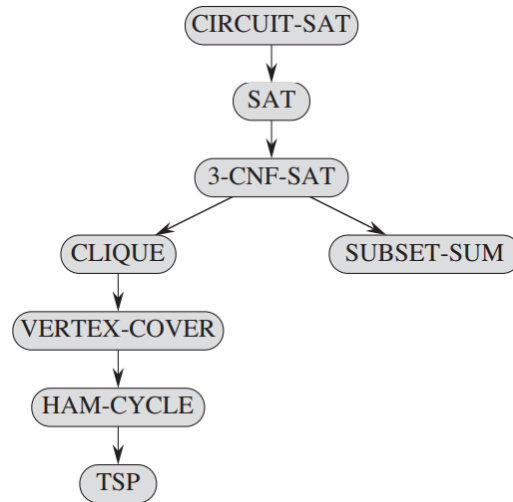
2-CNF satisfiability vs. 3-CNF satisfiability

Showing problem to be NP-Complete

1. Decision problems vs. optimization problems
2. Reductions



3. First NP-Complete Problem



P Problem

Problems that are solvable and verifiable in polynomial time. Binary search, Merge-sorting, Min- Max finding, Dijkstra's shortest path algorithm, Kruskal's algorithm all are p-class algorithms i.e all the deterministic polynomial algorithms.

NP Problems

A problem is called NP (nondeterministic polynomial) if its solution can be guessed and verified in polynomial time; nondeterministic means that no particular rule is followed to make the guess. TSP, Graph Coloring, SAT etc are NP problems i.e Non-Deterministic polynomial algorithms.

NP- Complete Problems

If a problem is NP and all other NP problems are polynomial-time reducible to it, the problem is NP-complete i.e

A language $L \subseteq \{0, 1\}^*$ is NP-complete if

- 1. $L \in \text{NP}$, and**
- 2. $L' \leq_p L$ for every $L' \in \text{NP}$**

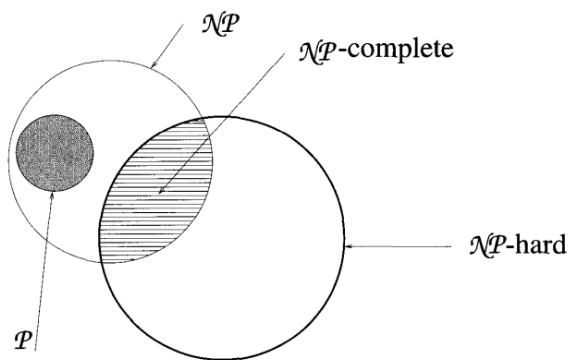
Some examples of this class are: Traveling salesman problem, satisfiability problems, and vertex-cover problems.

NP Hard Problems

If a language L satisfies above property 2, but not necessarily property 1, we say that L is NP-hard.

Example Hamiltonian cycle

NP Completeness and Problem Reducibility



Commonly believed relationship among P, NP, NP- complete, and NP-hard problems

Cooks Theorem (Without Proof, only Concept).

Statement: SAT is in P if and only if $P = NP$ or Satisfiability (SAT) is NP-Complete.

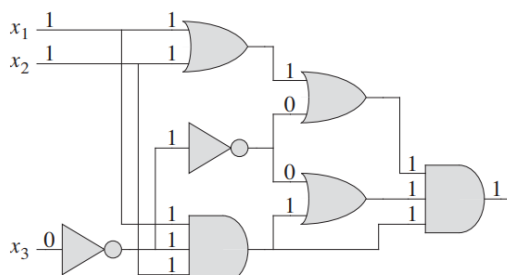
Cook and Levin in 1973 independently proved that the satisfiability problem (SAT) is NP-complete. Cook, in 1971, published an important paper titled '*The complexity of Theorem Proving Procedures*', in which he outlined the way of obtaining the proof of an NP-complete problem by reducing it to SAT. He proved **Circuit-SAT** and **3CNF-SAT** problems are as hard as SAT. Later, Karp reduced 21 optimization problems to the SAT, including Hamiltonian tour, vertex cover, clique, and proved that those problems are NP-complete.

SAT Problem: Given a boolean expression F having n variables $x_1, x_2, \dots, x_n$, and Boolean operators, is it possible to have an assignment for variables true or false such that binary expression F is true?

SAT is one of the toughest problems, as there is no known algorithm other than the brute force approach. A brute force algorithm would be an exponential-time algorithm, as 2^n possible assignments need to be tried to check whether the given Boolean expression is true or not.

Variation of SAT Problems

CIRCUIT-SAT: Boolean circuit, which is a collection of gates such as AND, OR, and NOT, and n inputs, is there any input assignments of Boolean variables so that the output of the given circuit is true? Again, the difficulty with these problems is that for n inputs to the circuit. 2^n possible outputs should be checked. Therefore, this brute force algorithm is an exponential algorithm and hence this is a hard problem.



$$f(x_1, x_2, x_3) = (x_1 + x_2 + x_3)(x_1 x_2 x_3' + x_3)(x_1 x_2 x_3')$$

Use

$$1. (p+qr) = (p+q)(p+r) \quad \text{And}$$

$$2. pqr = (p+qq')(q+pp')(r+qq')$$

For CIRCUIT-SAT to 3 CNF SAT

$$f(x_1, x_2, x_3) = (x_1 + x_2 + x_3)(x_1 + x_2 + x_3')(x_1 + x_2' + x_3)(x_1 + x_2' + x_3')(x_1' + x_2 + x_3)(x_1' + x_2 + x_3')(x_1' + x_2' + x_3)$$

CNF-SAT:

This problem is a restricted problem of SAT, where the expression should be in a conjunctive normal form (CNF). An expression is said to be in a conjunction form if all the clauses are connected by the Boolean AND

operator. Like in case of a **SAT**, this is about assigning truth values to n variables such that the output of the expression is true.

$$(x_1 \vee x_2 \vee \dots \vee x_n) \wedge (y_1 \vee x_2 \vee \dots \vee x_n) \wedge (x_1 \vee y_2 \vee \dots \vee x_n) \wedge (y_1 \vee y_2 \vee \dots \vee x_n) \wedge \dots \wedge (x_1 \vee x_2 \vee \dots \vee y_n) \wedge (y_1 \vee x_2 \vee \dots \vee y_n) \wedge (x_1 \vee y_2 \vee \dots \vee y_n) \wedge (y_1 \vee y_2 \vee \dots \vee y_n);$$

3-CNF SAT/ 3- SAT:

This problem is another variant where the additional restriction is that the expression is in a conjunctive normal form (CNF) and that every clause should contain **exactly three literals**. This problem is also about assigning n assignments of truth values to **n variables** of the Boolean expression such that the output of the expression is true.

Example $(x \vee \neg x \vee y) \wedge (\neg x \vee \neg y \vee y) \wedge (\neg x \vee y \vee z)$

Proof of NP Completeness

As in the procedure and definition of NP -Completeness in above, following three problems can be declared as NP-Complete.

CNF-SAT Problem

First we show CNF SAT is NP how ? (by showing this as decision or optimization problem)

Second Polynomial reduction from any known first NP problem i.e **Circuit SAT** $\rightarrow$ **CNF-SAT**,

Vertex Cover Problem

First we show Vertex Cover is in NP how ? (by showing this as decision or optimization problem)

Second Polynomial reduction from any known first NP problem i.e **Clique** $\rightarrow$ **Vertex Cover**

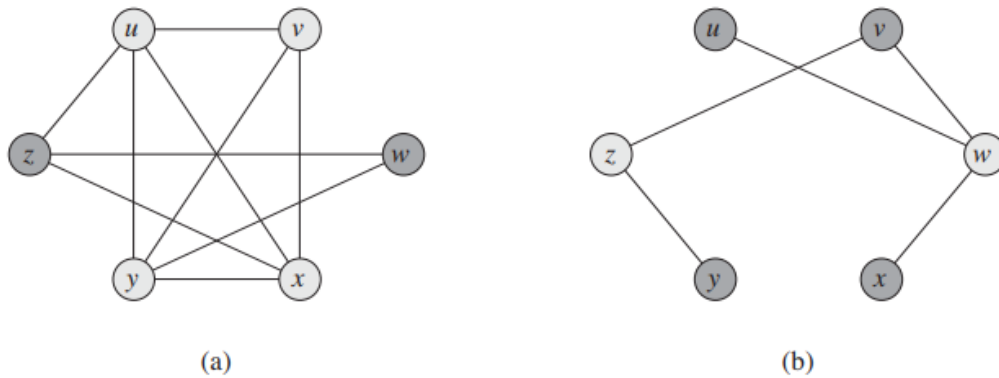


Figure Reducing CLIQUE to VERTEX-COVER. (a) An undirected graph $G = (V, E)$ with clique $V' = \{u, v, x, y\}$. (b) The graph G produced by the reduction algorithm that has vertex cover $V-V' = \{w, z\}$.

A **Clique** is defined as a subgraph of graph such that **every two distinct vertices in the clique are adjacent**, forming a **complete** subgraph. **Objective is to find maximum size Clique!**

Vertex Cover is a **subset** of vertices in a graph such that every edge of the graph is incident to at least one vertex in this subset.

Objective is to find the minimum size of Vertex-Cover!

Subset-Sum Problem

First, we show subset-Sum is in NP how? (by showing this as decision or optimization problem)

Second Polynomial reduction from any known first NP problem i.e 3-CNF SAT ->p Subset-Sum Problem

		x1	x2	x3	C1	C2	C3	C4
v1	=	1	0	0	1	0	0	1
v'1	=	1	0	0	0	1	1	0
v2	=	0	1	0	0	0	0	1
v'2	=	0	1	0	1	1	1	0
v3	=	0	0	1	0	0	1	1
v'3	=	0	0	1	1	1	0	0
s1	=	0	0	0	1	0	0	0
s'1	=	0	0	0	2	0	0	0
s2	=	0	0	0	0	1	0	0
s'2	=	0	0	0	0	2	0	0
s3	=	0	0	0	0	0	1	0
s'3	=	0	0	0	0	0	2	0
s4	=	0	0	0	0	0	0	1
s'4	=	0	0	0	0	0	0	2
t	=	1	1	1	4	4	4	4

Figure The reduction of 3-CNF-SAT to SUBSET-SUM

The formula in 3-CNF is $\phi = C_1 \wedge C_2 \wedge C_3 \wedge C_4$, where
 $C_1 = (X_1 \vee \neg X_2 \vee \neg X_3)$,
 $C_2 = (\neg X_1 \vee \neg X_2 \vee \neg X_3)$,
 $C_3 = (\neg X_1 \vee \neg X_2 \vee X_3)$, and
 $C_4 = (X_1 \vee X_2 \vee X_3)$. A satisfying assignment of ϕ is $(x_1 = 0, x_2 = 0, x_3 = 1)$. The set S produced by the reduction consists of the base-10 numbers shown; reading from top to bottom, $S = \{1001001, 1000110, 100001, 101110, 10011, 11100, 1000, 2000, 100, 200, 10, 20, 1, 2\}$. The target t is 1114444. The subset $S' \subseteq S$ is lightly shaded, and it contains v'_1, v'_2 , and v_3 , corresponding to the satisfying assignment. It also contains slack variables $s_1, s'_1, s'_2, s_3, s_4$, and s'_4 to achieve the target value of 4 in the digits labeled by C_1 through C_4 .

Approximation Algorithms[1Hr]

Concept and Application

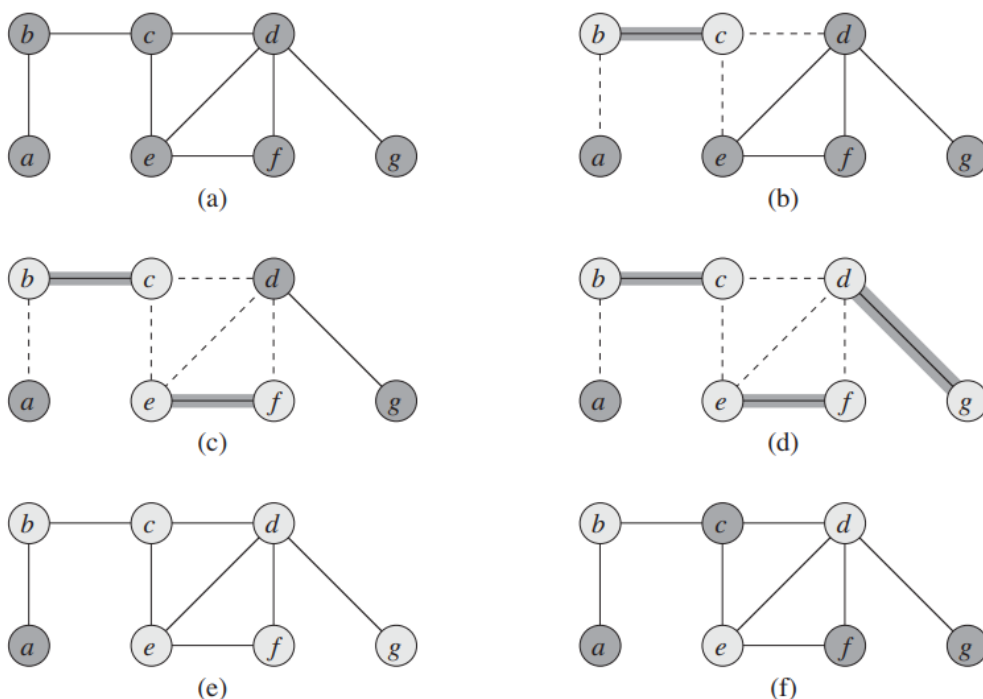
Many problems of practical significance are NP-complete, yet they are too important to abandon merely because we don't know how to find an optimal solution in polynomial time. Even if a problem is NP-complete, there may be hope. We have at least three ways to get around NP-completeness.

- First, if the actual inputs are small, an algorithm with exponential running time may be perfectly satisfactory.
- Second, we may be able to isolate important special cases that we can solve in polynomial time.
- Third, we might come up with approaches to find near-optimal solutions in polynomial time (either in the worst case or the expected case).

In practice, near-optimality is often good enough.

We call an algorithm that returns near-optimal solutions an approximation algorithm.

Vertex Cover Problem



The operation of APPROX-VERTEX-COVER

(a) The input graph G , which has 7 vertices and 8 edges. (b) The edge (b, c) , shown heavy, is the first edge chosen by APPROX-VERTEX-COVER. Vertices b and c , shown lightly shaded, are added to the set C containing the vertex cover being created. Edges (a, b) , (c, e) , and (c, d) , shown dashed, are removed since they are now covered by some vertex in C . (c) Edge (e, f) is chosen; vertices e and f are added to C . (d) Edge (d, g) is chosen; vertices d and g are added to C . (e) The set C , which is the vertex cover produced by APPROX-VERTEX-COVER, contains the six vertices b, c, d, e, f, g . (f) The optimal vertex cover for this problem contains only three vertices: b, d , and e .

APPROX-VERTEX-COVER(G)

```

1   $C = \emptyset$ 
2   $E' = G.E$ 
3  while  $E' \neq \emptyset$ 
4      let  $(u, v)$  be an arbitrary edge of  $E'$ 
5       $C = C \cup \{u, v\}$ 
6      remove from  $E'$  every edge incident on either  $u$  or  $v$ 
7  return  $C$ 
```


Subset Sum Problem

This decision problem asks whether there exists a subset of S that adds up exactly to the target value t . As we know this problem is NP-complete. The optimization problem associated with this decision problem arises in practical applications. In the optimization problem, we wish to find a subset of $\{x_1; x_2; \dots; x_n\}$ whose sum is as large as possible but not larger than t . For example, we may have a truck that can carry no more than t pounds, and n different boxes to ship, the i^{th} of which weighs x_i pounds. We wish to fill the truck with as heavy a load as possible without exceeding the given weight limit.

EXACT-SUBSET-SUM(S, t)

```
1   $n = |S|$ 
2   $L_0 = \langle 0 \rangle$ 
3  for  $i = 1$  to  $n$ 
4       $L_i = \text{MERGE-LISTS}(L_{i-1}, L_{i-1} + x_i)$ 
5      remove from  $L_i$  every element that is greater than  $t$ 
6  return the largest element in  $L_n$ 
```

To see how EXACT-SUBSET-SUM works, let P_i denote the set of all values obtained by selecting a (possibly empty) subset of $\{x_1; x_2; \dots; x_i\}$ and summing its members. **For example, if $S = \{1, 4, 5\}$, and $t=9$ then**

$L_1 = \{0, 1\};$
 $L_2 = \{0, 1, 4, 5\};$
 $L_3 = \{0, 1, 4, 5, 6, 9, 10\};$
 $L_3 = \{0, 1, 4, 5, 6, 9\};$

Given the identity: $P_i = p_{i-1} \cup (P_{i-1} + x_i)$,

Fully polynomial-time approximation scheme can be done for the subset-sum problem by “trimming” each list L_i after it is created. Trimming parameter d such that $0 < d < 1$, when we trim a list L by d , we remove as many elements from L as possible, in such a way that if L' is the result of trimming L , then for every element y that was removed from L , there is an element z still in L' that approximates y ,

$$\text{that is, } \frac{y}{1+d} \leq z \leq y$$

TRIM(L, δ)

```
1  let  $m$  be the length of  $L$ 
2   $L' = \langle y_1 \rangle$ 
3   $last = y_1$ 
4  for  $i = 2$  to  $m$ 
5      if  $y_i > last \cdot (1 + \delta)$       //  $y_i \geq last$  because  $L$  is sorted
6          append  $y_i$  onto the end of  $L'$ 
7           $last = y_i$ 
8  return  $L'$ 
```

For example, if $\delta = 0.1$ and

$L = \langle 10, 11, 12, 15, 20, 21, 22, 23, 24, 29 \rangle$,

then we can trim L to obtain

$L' = \langle 10, 12, 15, 20, 23, 29 \rangle$,

$12 \cong 11$ // 12 is approximate to 11

$20 \cong 21$ and 22 // 20 is approximate to 21 and 22

23 $\cong$ 24 // 23 is approximate to 24

Now with trimming function algorithm reduces to polynomial

APPROX-SUBSET-SUM(S, t, ϵ)

```
1   $n = |S|$ 
2   $L_0 = \langle 0 \rangle$ 
3  for  $i = 1$  to  $n$ 
4       $L_i = \text{MERGE-LISTS}(L_{i-1}, L_{i-1} + x_i)$ 
5       $L_i = \text{TRIM}(L_i, \epsilon/2n)$ 
6      remove from  $L_i$  every element that is greater than  $t$ 
7  let  $z^*$  be the largest value in  $L_n$ 
8  return  $z^*$ 
```

Where $0 < \epsilon < 1$